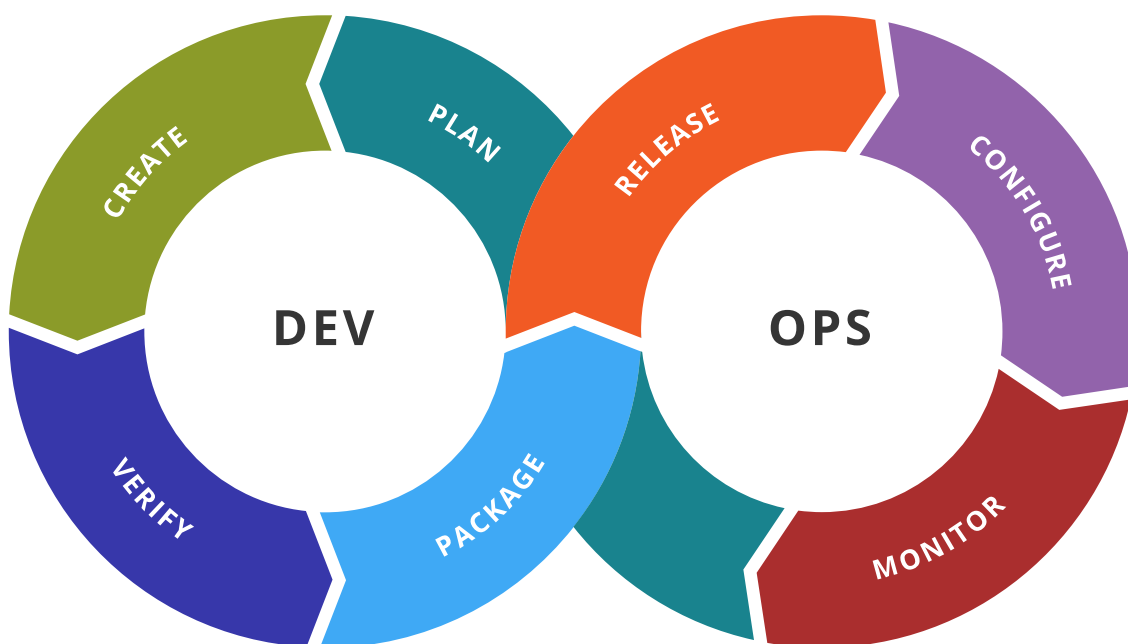


Что такое DevOps?

Прежде чем мы начнем работу с Docker, давайте заложим основу для изучения правильного мышления. Определение DevOps — не тривиальная задача, но сам термин состоит из двух частей: *Dev* и *Ops*. *Dev* относится к разработке программного обеспечения, а *Ops* — к операциям. Простое определение DevOps заключается в том, что это означает, что выпуск, настройка и мониторинг программного обеспечения находятся в руках тех же людей, которые его разрабатывают.

Более формальное определение предлагается [Jabbari et al.](#): "DevOps — это методология разработки, направленная на преодоление разрыва между разработкой и операциями, с акцентом на коммуникацию и сотрудничество, непрерывную интеграцию, обеспечение качества и доставку с автоматизированным развертыванием с использованием набора практик разработки".



Изображение цепочки инструментов DevOps от Kharnagy из [википедии](#)

Иногда DevOps рассматривается как роль, который может выполнять один человек или команда. Вот некоторые внешняя мотивация для изучения навыков DevOps: [Зарплата по типу разработчика в опросе StackOverflow](#). Вы не станете специалистом по DevOps только благодаря этому курсу, но получите навыки, которые помогут вам ориентироваться в мире, где контейнеризация становится все более распространенной.

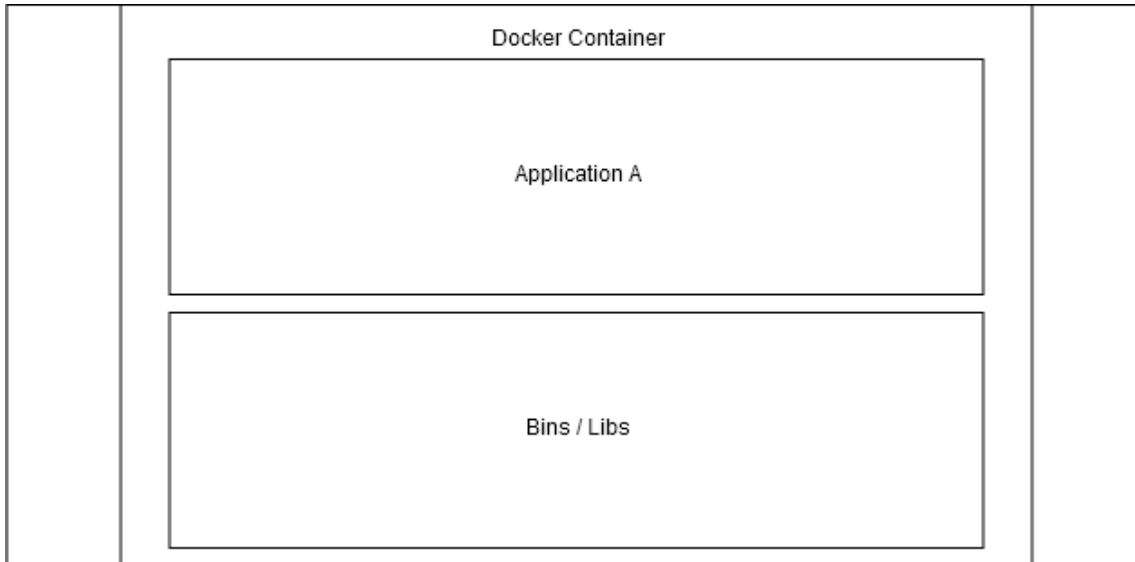
В течение этого курса мы сосредоточимся в основном на упаковке, выпуске и настройке приложений. Вас не попросят планировать или создавать новое программное обеспечение. Мы рассмотрим Docker и несколько технологий, которые вы можете встретить в повседневной жизни, включая, например, Redis и Postgres. См. [опрос StackOverflow](#), насколько тесно они коррелируют с этими технологиями.

Что такое Docker?

"Docker — это набор продуктов платформы как услуги (PaaS), которые используют виртуализацию на уровне ОС для доставки программного обеспечения в пакетах, называемых контейнерами." — [из Википедии](#).

Итак, убрав жаргон, мы получаем два определения:

1. Docker — это набор инструментов для доставки программного обеспечения в контейнерах.
2. Контейнеры — это пакеты программного обеспечения.



Изображение выше иллюстрирует, как контейнеры включают приложение и его зависимости. Эти контейнеры изолированы так, что они не мешают друг другу или программному обеспечению, работающему вне контейнеров. Если вам нужно взаимодействовать с ними или включить взаимодействие между ними, Docker предлагает инструменты для этого.

Преимущества от контейнеров

Контейнеры упаковывают приложения. Звучит просто, правда? Чтобы проиллюстрировать потенциальные преимущества, давайте поговорим о различных сценариях.

Сценарий 1: Работает на моей машине

Давайте сначала более подробно рассмотрим, что происходит в веб-разработке без контейнеров, следуя цепочке выше, начиная с "План".

Сначала вы планируете приложение. Затем ваша команда из 1-н разработчиков создает программное обеспечение. Оно работает на вашем компьютере. Оно может даже пройти через тестовый конвейер, работая идеально. Вы отправляете его на сервер и...

...оно не работает.

Это известно как проблема **"работает на моей машине"**. Единственный способ решить эту проблему — выяснить, что, черт возьми, разработчик установил на своей машине, что заставило приложение работать.

Контейнеры решают эту проблему, позволяя разработчику лично запускать приложение внутри контейнера, который затем включает все зависимости, необходимые для работы приложения.

- Вы все еще можете иногда слышать о проблемах "работает в моем **контейнере**" — это часто просто ошибки использования.

Сценарий 2: Изолированные среды

У вас есть 5 различных приложений Python. Вам нужно развернуть их на сервере, на котором уже есть приложение, требующее Python 2.7, и, конечно, ни одно из ваших приложений не использует 2.7. Что вы делаете?

Поскольку контейнеры упаковывают программное обеспечение со всеми его зависимостями, вы упаковываете существующее приложение и все 5 новых с их соответствующими версиями Python — и все.

Я могу только представить катастрофу, которая произойдет, если вы попытаетесь запустить их бок о бок на одной машине без изоляции сред. Это звучит больше как бомба замедленного действия. Иногда различные части системы могут меняться со временем, что может привести к тому, что приложение перестанет работать. Эти изменения могут быть чем угодно, от обновления операционной системы до изменений в зависимостях.

Сценарий 3: Разработка

Вас привлекают в команду разработчиков. Они запускают веб-приложение, которое использует другие сервисы при работе: базу данных Postgres, MongoDB, Redis и ряд других. Достаточно просто, вы устанавливаете все, что требуется для запуска приложения, и всех приложений, от которых оно зависит...

Какая головная боль — начать устанавливать, а затем управлять базами данных разработки на своей собственной машине.

К счастью, к тому времени, когда вам говорят сделать это, вы уже эксперт по Docker. С одной командой вы получаете изолированное приложение, например Postgres или Mongo, работающее на вашей машине.

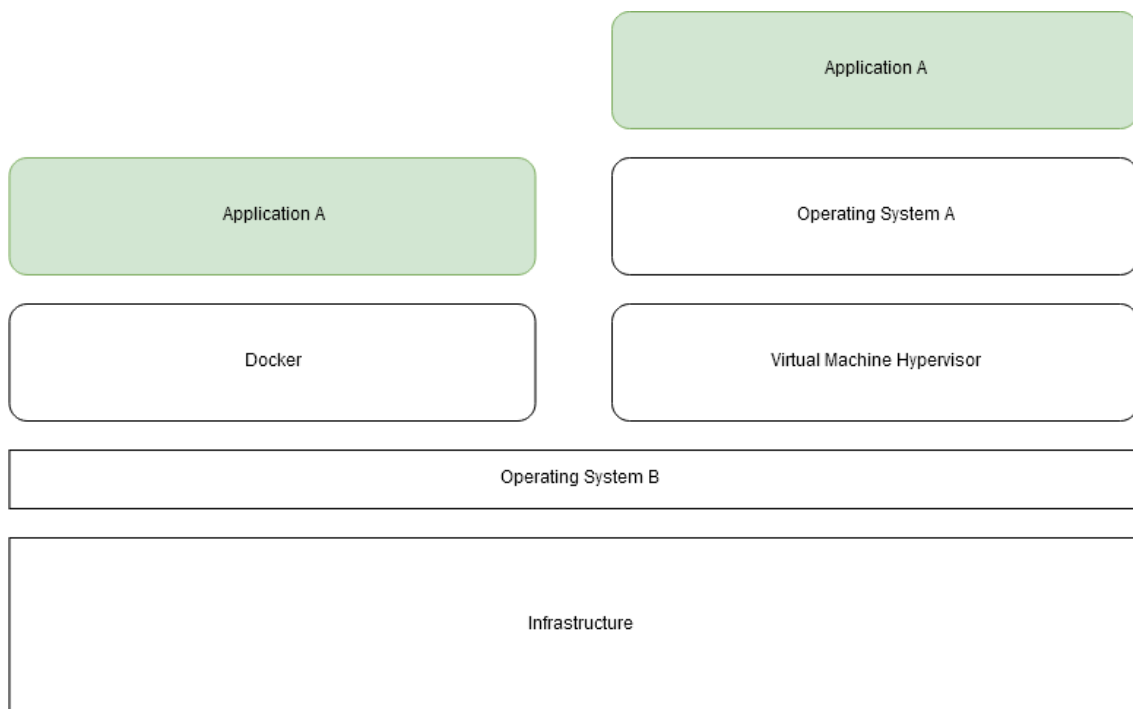
Сценарий 4: Масштабирование

Запуск и остановка Docker контейнера требует небольших накладных расходов. Но когда вы запускаете свой собственный Netflix или Facebook, вы хотите соответствовать меняющемуся спросу. С помощью некоторых продвинутых инструментов, которые мы изучим в частях 2 и 3, мы можем мгновенно запускать несколько контейнеров и балансировать трафик между ними.

Оркестрация контейнеров будет обсуждаться в частях 2 и 3. Но простейший пример: что происходит, когда одно приложение умирает? Система оркестрации замечает это, распределяет трафик между работающими репликами и запускает новый контейнер для замены мертвого.

Виртуальные машины

Разве уже нет решения для этого? Виртуальные машины — это не то же самое, что контейнеры — они решают разные проблемы. Мы не будем рассматривать виртуальные машины в этом курсе. Однако вот диаграмма, чтобы дать вам общее представление о различии.



Виртуальные машины (VM) работают на [гипервизоре](#), который виртуализирует физическое оборудование. Каждая VM включает полную операционную систему (ОС) вместе с необходимыми бинарными файлами и библиотеками, что делает их более тяжелыми и требовательными к ресурсам. Контейнеры, с другой стороны, разделяют ядро хостовой ОС и упаковывают только приложение и его зависимости, что приводит к более легкому и эффективному решению.

VM обеспечивают сильную изоляцию и подходят для запуска нескольких сред ОС, но они имеют накладные расходы на производительность и более длительное время запуска. Контейнеры предлагают более быстрый запуск, лучшее использование ресурсов и высокую переносимость между различными средами, хотя их изоляция находится на уровне процесса, что может быть не так надежно, как у VM. В целом, VM можно использовать для сценариев, требующих полных сред ОС, в то время как контейнеры превосходят в легком, эффективном и согласованном развертывании приложений.

Docker полагается на ядра Linux, что означает, что macOS и Windows не могут запускать Docker нативно без некоторых дополнительных шагов. Каждая операционная система имеет свое собственное решение для запуска Docker. Например, Docker для Mac на самом деле использует виртуальную машину, которая запускает экземпляр Linux, внутри которого работает Docker.

Запуск контейнеров

У вас уже установлен Docker, так что давайте запустим наш первый контейнер!

hello-world — это простое приложение, которое выводит "Hello from Docker!" и некоторую дополнительную информацию.

Просто запустите `docker container run hello-world`, вывод будет следующим:

```
$ docker container run hello-world
Unable to find image 'hello-world:latest' locally
```

```
latest: Pulling from library/hello-world
b8dfde127a29: Pull complete
Digest: sha256:308866a43596e83578c7dfa15e27a73011bdd402185a84c5cd7f32a88b501a24
Status: Downloaded newer image for hello-world:latest
```

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

To try something more ambitious, you can run an Ubuntu container with:

```
$ docker run -it ubuntu bash
```

Share images, automate workflows, and more with a free Docker ID:
<https://hub.docker.com/>

For more examples and ideas, visit:
<https://docs.docker.com/get-started/>

Команда `docker container run` имеет сокращенную форму `docker run`, так что вместо

```
$ docker container run hello-world
```

мы могли бы сделать то же самое с меньшим количеством набора и использовать просто

```
$ docker run hello-world
```

Если вы уже запускали hello-world ранее, он пропустит первые 5 строк. Первые 5 строк говорят, что **образ** "hello-world:latest" не был найден и был загружен. Попробуйте снова:

```
$ docker run hello-world
```

```
Hello from Docker!
```

```
...
```

Он нашел **образ** локально, поэтому сразу перешел к запуску hello-world.

:::tip Напоминание о безопасности: Загрузка из интернета

Имейте в виду, что мы загружаем вещи из интернета. Двойная проверка того, что вы запускаете, всегда хорошая идея.

...

Итак, что такое образ?

Образ и контейнеры

Поскольку мы уже знаем, что такое контейнеры, легче объяснить образы через них: Контейнеры — это экземпляры образов. Базовая ошибка — путать образы и контейнеры.

Кулинарная метафора:

Думайте о контейнере как о готовом к употреблению блюде, которое вы можете просто разогреть и съесть. Образ, с другой стороны, — это рецепт и ингредиенты для этого блюда.

Так что, как вам нужен рецепт и ингредиенты, чтобы приготовить блюдо, вам нужен образ и среда выполнения контейнеров (движок Docker), чтобы создать контейнер. Образ предоставляет все необходимые инструкции и зависимости для работы контейнера, как рецепт предоставляет шаги и ингредиенты для приготовления блюда.

Короче говоря, образ похож на чертеж или шаблон и строительный материал, а контейнер — это экземпляр этого чертежа или шаблона.

Образ

Docker образ — это файл. Образ никогда не меняется; вы не можете редактировать существующий файл. Создание нового образа происходит путем запуска из базового образа и добавления к нему новых **слоев**. Мы поговорим о слоях позже, но вы должны думать об образах как о *неизменяемых*, они не могут быть изменены после создания.

Просмотрите все свои образы с помощью `docker image ls`

```
$ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
hello-world	latest	d1165f221234	9 days ago	13.3kB

Контейнеры создаются из образов, поэтому когда мы запускали hello-world дважды, мы загрузили один образ и создали два контейнера из одного образа.

Ну а если образы используются для создания контейнеров, откуда берутся образы? Этот файл образа создается из инструкционного файла с именем **Dockerfile**, который анализируется при выполнении `docker image build`.

Dockerfile — это файл, который по умолчанию называется *Dockerfile*, который выглядит примерно так

Dockerfile

```
FROM <image>:<tag>

RUN <install some dependencies>

CMD <command that is executed on `docker container run`>
```

и представляет собой набор инструкций для сборки образа. Мы рассмотрим Dockerfiles позже, когда начнем создавать свои собственные образы.

Если вернуться к кулинарной метафоре, поскольку Dockerfile предоставляет инструкции, необходимые для сборки образа, вы можете думать об этом как о рецепте для образов. Теперь мы на 2 рецепта глубже, так как Dockerfile — это рецепт для образа, а образ — это рецепт для контейнера. Единственное различие в том, что Dockerfile написан нами, тогда как образ написан нашей машиной на основе Dockerfile!

Контейнер

Контейнеры содержат только то, что требуется для выполнения приложения; и вы можете запускать, останавливать и взаимодействовать с ними. Они являются **изолированными** средами на хост-машине с возможностью взаимодействовать друг с другом и с хост-машиной через определенные методы (TCP/UDP).

Посмотрите все свои контейнеры с помощью `docker container ls`

```
$ docker container ls
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Без флага `-a` он будет выводить только запущенные контейнеры. `hello-world`, которые мы запускали, уже завершились.

```
$ docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
b7a53260b513	hello-world	"/hello"	5 minutes ago	Exited (0) 5 minutes ago
1cd4cb01482d	brave_bhabha	"/hello"	8 minutes ago	Exited (0) 8 minutes ago
	vibrant_bell			

Команда `docker container ls` также имеет более короткую форму `docker ps`, которую предпочитают многие, так как она требует гораздо меньше набора...

Основы Docker CLI

Мы используем командную строку для взаимодействия с "Docker Engine", который состоит из 3 частей: CLI, REST API и Docker daemon. Когда вы выполняете команду, например, `docker container run`, за кулисами клиент отправляет запрос через REST API в **Docker daemon**, который заботится об образах, контейнерах и других ресурсах.

Вы можете прочитать [документацию для получения дополнительной информации](#). Но даже если вы найдете более 50 команд в документации, только несколько из них нужны для общего использования. В конце этого раздела есть список наиболее часто используемых базовых команд.

Одна из них уже знакома: `docker container run <image>`, которая инструктирует daemon создать контейнер из образа и загрузить образ, если он недоступен.

Давайте удалим образ, так как он нам больше не нужен, `docker image rm hello-world` звучит правильно. Однако это должно завершиться ошибкой:

```
$ docker image rm hello-world
Error response from daemon: conflict: unable to remove repository reference
```

```
"hello-world" (must force) - container <container ID> is using its referenced image
<image ID>
```

Это означает, что контейнер, созданный из образа *hello-world*, все еще существует, и удаление *hello-world* может иметь последствия. Поэтому перед удалением образов вы должны сначала удалить ссылающийся контейнер. Принуждение обычно плохая идея, особенно пока мы все еще учимся.

Запустите `docker container ls -a`, чтобы снова просмотреть все контейнеры.

```
$ docker container ls -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
PORTS NAMES				
b7a53260b513	hello-world	"/hello"	35 minutes ago	Exited (0) 35 minutes ago
		brave_bhabha		
1cd4cb01482d	hello-world	"/hello"	41 minutes ago	Exited (0) 41 minutes ago
		vibrant_bell		

Обратите внимание, что контейнеры имеют *CONTAINER ID* и *NAME*. Имена в настоящее время генерируются автоматически. Когда у нас много разных контейнеров, мы можем использовать `grep` (или другую подобную утилиту) для фильтрации списка:

```
$ docker container ls -a | grep hello-world
```

Давайте удалим контейнер с помощью команды `docker container rm`. Она принимает имя или ID контейнера в качестве аргументов.

Обратите внимание, что команда также работает с первыми несколькими символами ID. Например, если ID контейнера — `3d4bab29dd67`, вы можете использовать `docker container rm 3d`, чтобы удалить его. Использование сокращения для ID не удалит несколько контейнеров, так что если у вас есть два ID, начинающихся с `3d`, будет выведено предупреждение, и ни один не будет удален. Вы также можете использовать несколько аргументов: `docker container rm id1 id2 id3`

Если у вас есть сотни остановленных контейнеров, и вы хотите удалить их все, вы должны использовать `docker container prune`. `Prune` также можно использовать для удаления "висячих" образов с помощью `docker image prune`. Висячие образы — это образы, которые не имеют имени и не используются. Они могут быть созданы вручную и автоматически генерируются во время сборки. Их удаление просто экономит немного места.

И наконец, вы можете использовать `docker system prune`, чтобы очистить почти все. Мы еще не знакомы с исключениями, которые `docker system prune` не удаляет.

После удаления всех контейнеров *hello-world* запустите `docker image rm hello-world`, чтобы удалить образ. Вы можете использовать `docker image ls`, чтобы подтвердить, что образ не отображается.

Вы также можете использовать команду `image pull`, чтобы загружать образы без их запуска: `docker image pull hello-world`

Давайте попробуем запустить новый контейнер:

```
$ docker run nginx
```


С некоторыми контейнерами командная строка, похоже, зависает после загрузки и запуска контейнера. Это может быть потому, что этот конкретный контейнер теперь работает в текущем терминале, блокируя ввод. Вы можете наблюдать это с помощью `docker container ls` из другого терминала. В этой ситуации можно выйти, нажав `control + c`, и попробовать снова с флагом `-d`.

```
$ docker run -d nginx
c7749cf989f61353c1d433466d9ed6c45458291106e8131391af972c287fb0e5
```

Флаг `-d` запускает контейнер *detached*, то есть он работает в фоновом режиме. Контейнер можно увидеть с помощью

```
$ docker container ls
CONTAINER ID   IMAGE     COMMAND                  CREATED
STATUS        PORTS     NAMES
c7749cf989f6   nginx    "nginx -g 'daemon of..." 35 seconds ago
Up 34 seconds  80/tcp    blissful_wright
```

Теперь, если мы попытаемся удалить его, это не сработает:

```
$ docker container rm blissful_wright
Error response from daemon: You cannot remove a running container
c7749cf989f61353c1d433466d9ed6c45458291106e8131391af972c287fb0e5. Stop the container
before attempting removal or force remove
```

Мы должны сначала остановить контейнер, используя `docker container stop blissful_wright`, а затем использовать `rm`.

Принуждение также возможно, и мы можем безопасно использовать `docker container rm --force blissful_wright` в этом случае. Опять же, для обоих вместо имени мы могли бы использовать ID или его части, например, `c77`.

Со временем Docker daemon обычно забивается старыми образами и контейнерами.

Наиболее используемые команды

команда	объяснение	сокращение
<code>docker image ls</code>	Показывает все образы	<code>docker images</code>
<code>docker image rm <image></code>	Удаляет образ	<code>docker rmi</code>
<code>docker image pull <image></code>	Загружает образ из docker registry	<code>docker pull</code>
<code>docker container ls -a</code>	Показывает все контейнеры	<code>docker ps -a</code>
<code>docker container run <image></code>	Запускает контейнер из образа	<code>docker run</code>
<code>docker container rm <container></code>	Удаляет контейнер	<code>docker rm</code>
<code>docker container stop <container></code>	Останавливает контейнер	<code>docker stop</code>
<code>docker container exec <container></code>	Выполняет команду внутри контейнера	<code>docker exec</code>

Для всех них контейнер может быть либо ID контейнера, либо именем контейнера. То же самое для образов. В будущем мы можем использовать сокращения в материалах.

Некоторые сокращения — это устаревшие версии того же самого. Вы можете использовать любое.

Упражнения 1.1-1.2

:::info Упражнение 1.1: Начало работы

Поскольку мы уже сделали "Hello, World!" в материале, давайте сделаем что-то другое.

Запустите 3 контейнера из образа, который не завершается автоматически (такого как `nginx`) в `detached` режиме.

Остановите два контейнера и оставьте один контейнер работающим.

Отправьте вывод для `docker ps -a`, который показывает 2 остановленных контейнера и один работающий.

:::

:::info Упражнение 1.2: Очистка

У нас есть контейнеры и образ, которые больше не используются и занимают место. Запуск `docker ps -a` и `docker image ls` подтвердит это.

Очистите Docker daemon, удалив все образы и контейнеры.

Отправьте вывод для `docker ps -a` и `docker image ls`

:::